

Learned Query Optimization in Modern Database Systems: A Comprehensive Survey

Tinon Turja Majumder

Department of Computer Science and Engineering
United International University, Dhaka, Bangladesh
Email: tmajumder2410048@mscse.uui.ac.bd

Abstract—Query optimization remains a cornerstone of relational database performance but continues to face limitations due to inaccurate cost estimation, static heuristics, and poor adaptability across workloads and hardware environments. The emergence of machine learning has given rise to *Learned Query Optimization* (LQO), which replaces or augments traditional rule- and cost-based approaches with data-driven learning models. This survey provides a comprehensive review of the evolution, design paradigms, and implementation strategies of learned query optimizers, synthesizing findings from over twenty state-of-the-art systems, including Neo, Bao, Balsa, Lero, LEON, Kepler, LOGER, and their successors. We present a structured taxonomy of LQO techniques—spanning learned cost models, learning-to-rank optimizers, reinforcement learning frameworks, and hybrid hint-guided systems—and evaluate their comparative performance, robustness, and generalization capabilities. The paper further examines current challenges such as data labeling, reproducibility, and integration overhead, while identifying open research directions involving cross-engine transfer learning, uncertainty modeling, and continual adaptation. Finally, recent advances such as QO-GNN, DQO, and Morpheus demonstrate a shift toward graph-based, transferable, and self-supervising optimizers, marking a crucial step toward autonomous and intelligent database management systems.

Index Terms—Query optimization, machine learning, learned query optimizer, reinforcement learning, cost estimation, data-driven optimization, database management systems, adaptive optimization, deep learning, autonomous database systems.

I. INTRODUCTION

Query optimization is a fundamental component of every relational database management system (DBMS). Given a declarative SQL statement, the optimizer’s task is to determine the most efficient physical execution plan among an exponentially large number of alternatives. Historically, optimizers have relied on rule-based heuristics and cost-based frameworks derived from the pioneering System R project. While these mechanisms have matured over decades, they remain highly sensitive to inaccurate statistics, static assumptions, and hand-tuned parameters [1]–[3]. Consequently, even small errors in cardinality or cost estimation can cascade into orders-of-magnitude performance regressions.

Recent advances in machine learning (ML) have inspired a new generation of *learned query optimizers (LQOs)* that replace, enhance, or guide traditional components with data-driven models. Instead of manually specifying formulas for operator cost or join order search, LQOs learn these relationships from execution feedback. Systems such as Neo [1], Bao [4],

Balsa [5], and Lero [6] have demonstrated that optimizers can continuously adapt to workload and data shifts—something rule-based systems cannot easily achieve.

This survey provides a comprehensive synthesis of modern LQO research, classifying approaches by their learning paradigm, degree of optimizer integration, and robustness mechanisms. We review over twenty representative systems and proposals, highlighting their architectures, evaluation benchmarks, and deployment readiness. The goal is to bridge conceptual understanding with empirical insight so that database practitioners, system architects, and researchers can assess where learning truly advances optimization and where open problems remain.

The key contributions of this paper are threefold:

- We present a structured taxonomy of learned query optimization techniques, spanning cost modeling, ranking, reinforcement learning, and hybrid hint-guided systems.
- We compare representative implementations in terms of objectives, feature representations, training strategies, robustness, and real-world deployability.
- We identify cross-cutting challenges—robustness, reproducibility, and generalization—and outline future research opportunities toward autonomous, risk-aware optimizers.

The remainder of the paper is organized as follows. Section II reviews the evolution and limitations of traditional query optimization. Section III motivates the shift toward learning-based methods. Section IV presents a taxonomy of learned optimizers. Section V provides a detailed comparative analysis, followed by discussions on challenges (Section VI) and open research directions (Section VII). Section VIII concludes the paper.

II. BACKGROUND: TRADITIONAL QUERY OPTIMIZATION

A. Overview

Query optimization translates an SQL query into an internal algebraic representation and searches for the lowest-cost execution plan. Each plan is a combination of join orders, access paths, and physical operators such as nested-loop, hash, or merge joins. The number of possible plans grows super-exponentially with the number of relations, making exhaustive enumeration infeasible [2], [3]. Thus, practical optimizers employ pruning heuristics and cost models to estimate and compare alternative plans.

B. Cost-Based Optimization

The cost-based optimizer (CBO), introduced in System R, remains the dominant paradigm. It estimates the cost of each operator using a hand-crafted cost function that combines CPU, I/O, and memory components. A critical input is the *cardinality estimate*—the expected number of tuples produced at each stage. Cardinality is computed using statistical synopses such as histograms or samples, under assumptions of attribute independence and uniform distribution. However, these assumptions often break in real data, leading to large estimation errors. Modern DBMSs like PostgreSQL, SQL Server, and Oracle still rely on variants of this model [7].

C. Heuristic and Rule-Based Optimization

Before CBOs became standard, optimizers followed fixed heuristics: push selections down, apply projections early, and choose smaller joins first. Even cost-based systems still retain heuristic pruning (e.g., left-deep join trees, limited reordering) to control search explosion. While efficient, such rules can yield locally optimal but globally suboptimal plans, especially for complex analytic queries [8].

D. Limitations of Traditional Optimizers

Despite decades of refinement, traditional optimizers exhibit several enduring weaknesses:

- 1) **Inaccurate Cardinality Estimation:** Errors propagate through joins, sometimes by several orders of magnitude [7].
- 2) **Static Behavior:** Cost models and parameters are manually calibrated and rarely adapt automatically to data or workload changes.
- 3) **Limited Search Space:** Heuristics restrict plan exploration; exhaustive enumeration is computationally infeasible.
- 4) **Manual Tuning:** Each DBMS uses its own proprietary parameters and requires expert intervention.
- 5) **Lack of Robustness:** Plans optimized for one scenario may degrade drastically under new distributions or hardware [1], [4].

E. Transition Toward Learning

Recognizing these limitations, researchers began exploring learned alternatives capable of replacing fragile hand-crafted models with adaptive, data-driven predictors. Early successes in learned cardinality estimation [7] and cost modeling [1] suggested that machine learning could capture correlations missed by analytical models. These efforts paved the way for holistic learned optimizers, as detailed in the next sections.

III. MOTIVATION FOR LEARNED QUERY OPTIMIZATION

A. The Inherent Complexity of Query Optimization

Query optimization is a combinatorial search problem. For a query joining n relations, the number of possible join orders grows super-exponentially ($O(n!)$ for bushy trees). Even dynamic-programming optimizers must prune heavily

using heuristics, leaving many potentially better plans unexplored [2], [3]. Because classical optimizers cannot learn from prior executions, mistakes such as poor join order selection repeat across workloads.

Machine learning offers a data-driven solution: by learning from historical execution feedback, the optimizer can model complex interactions among predicates, data distributions, and operators—capturing relationships that hand-tuned heuristics miss.

B. The Cost-Estimation Bottleneck

Cost estimation is the most error-prone stage of optimization. Traditional models rely on simplifying independence assumptions and coarse statistics, which often mispredict operator cardinalities by several orders of magnitude [7]. Learned models such as Neo [1] and the Learned Cost Models for Query Optimization framework [2] employ neural or regression functions that map query-plan features directly to observed runtimes, substantially improving accuracy. Kepler [9] extends this idea to parametric query optimization by integrating uncertainty-aware Gaussian processes for robustness.

C. Toward Adaptive and Self-Tuning Systems

A major motivation is to enable DBMSs that continuously self-tune. Bao [4] and Balsa [5] exemplify this shift. Bao uses contextual bandits to choose optimizer hints per query, learning from immediate feedback. Balsa applies deep reinforcement learning (DRL) to construct join orders without expert supervision, optimizing long-term performance through rewards. Both demonstrate the viability of **self-improving** optimizers that adapt as workloads evolve.

D. Dynamic and Distributed Environments

Distributed databases further complicate optimization: network latency, data placement, and resource contention dominate runtime cost. Traditional static models struggle here, motivating intelligent optimizers that adapt to changing environments. LOGER [10] and intelligent distributed frameworks [11] incorporate probabilistic learning to evaluate plan robustness under uncertainty, adjusting execution strategies online.

E. Robustness and Generalization

Beyond raw accuracy, robustness across unseen workloads is crucial. Lero and LEON [6], [12] emphasize learning-to-rank formulations that compare plans pairwise rather than predicting absolute latency, improving generalization. Kepler’s uncertainty gating and LOGER’s graph-based plan encoding also address this challenge.

F. Summary

In short, learned optimization is driven by the need for (1) accuracy in cost estimation, (2) adaptivity to workload change, (3) robustness under uncertainty, and (4) automation of DBA-intensive tuning. The following section categorizes how current systems realize these objectives.

IV. TAXONOMY OF LEARNED QUERY OPTIMIZATION APPROACHES

Research on learned query optimization can be grouped into four major categories:

- 1) **Learned Cost Models (LCMs):** Replace analytical cost formulas with regression or neural predictors (e.g., Neo [1], Kepler [9]).
- 2) **Learning-to-Rank Optimizers:** Predict which of two candidate plans is faster instead of absolute latency (e.g., Lero [6], LEON [12], LOGER [10]).
- 3) **Reinforcement-Learning (RL) Optimizers:** Formulate plan construction as sequential decision-making (e.g., Balsa [5], LOGER [10]).
- 4) **Hybrid and Hint-Guided Optimizers:** Steer existing optimizers using lightweight learned components (e.g., Bao [4], FASTgres [10]).

Each family differs in its learning paradigm, integration depth, and robustness. Table I summarizes their defining characteristics.

A. Learned Cost Models (LCMs)

LCMs act as drop-in replacements for cost estimation. They use supervised learning to map plan features to observed runtimes, often outperforming analytical models when trained on sufficient data. Neo [1] pioneered this paradigm; Kepler [9] adds uncertainty quantification to avoid regressions.

B. Learning-to-Rank Optimizers

These methods learn relative plan preferences. By ranking candidate plans, they minimize sensitivity to absolute runtime noise and transfer better across workloads. Lero [6] introduced pairwise ranking, while LEON [12] adds pruning guided by model confidence.

C. Reinforcement-Learning Optimizers

RL-based systems treat optimization as sequential decision-making. Balsa [5] learns join-order policies using deep RL, improving after each feedback cycle. LOGER [10] augments RL with graph neural networks and beam search to improve robustness.

D. Hybrid and Hint-Guided Optimizers

Hybrid systems integrate ML without replacing the optimizer. Bao [4] uses contextual bandits to select hints, continuously adapting to feedback. FASTgres and related systems generalize this approach to larger workloads and multiple PostgreSQL versions.

E. Summary

These categories form a continuum: from component-level learning (LCMs) to full plan synthesis (RL). Hybrid hinting methods currently dominate deployments due to safety and ease of integration, while end-to-end RL optimizers push the boundary of automation.

V. COMPARATIVE ANALYSIS OF MAJOR SYSTEMS

To highlight the diversity of design philosophies among learned optimizers, this section presents a comparative analysis of major systems drawn from academic and industrial research. The comparison focuses on five key dimensions: learning objective, model type, integration depth, adaptivity, and robustness mechanisms.

A. Observations

From Table II, several trends emerge:

- **Hybrid designs (Bao, FASTgres)** dominate deployments due to easy integration and low risk.
- **End-to-end optimizers (Balsa, LOGER)** demonstrate automation potential but require heavy training infrastructure.
- **Ranking-based models (Lero, LEON)** provide strong generalization and robustness to noise.
- **Cost model replacements (Neo, Kepler)** yield immediate performance benefits but need reliable feature extraction pipelines.

Empirically, Bao and Kepler remain the most production-ready, while Balsa and LOGER push boundaries for fully learned optimizers.

VI. CHALLENGES AND LIMITATIONS

A. Data Collection and Labeling

Training learned optimizers requires accurate runtime measurements for diverse query-plan pairs. Collecting such data is time-consuming and may risk performance degradation in production systems. Synthetic workloads can mitigate this but often lack real-world diversity [2].

B. Generalization and Robustness

Models trained on specific schemas or workloads often fail under distributional shifts. Lero [6] and Kepler [9] address this through ranking formulations and uncertainty quantification, respectively, but generalization across heterogeneous data and hardware remains unsolved.

C. Integration Overhead

End-to-end frameworks like Balsa and LOGER require invasive modifications to DBMS internals. Hybrid systems such as Bao [4] are easier to deploy but offer limited control. Achieving deep integration while maintaining modularity is a continuing challenge.

D. Explainability and Debugging

Unlike analytical models, neural optimizers lack interpretability. Explaining why a model prefers one plan over another is difficult, hindering trust and debugging. Recent work on hybrid symbolic-neural cost models [2] and uncertainty calibration [9] partially alleviates this issue.

TABLE I: Taxonomy of Learned Query Optimization Approaches

Category	Core Learning Task	Representative Systems	Learning Paradigm	Key Advantages / Limitations
Learned Cost Models (LCMs)	Predict operator or plan cost	Neo, Kepler, Learned Cost Models	Supervised regression	Accurate cost prediction; needs large labeled data
Learning-to-Rank Optimizers	Rank candidate plans by performance	Lero, LEON, LOGER	Pairwise / listwise ranking	Robust to noise; requires diverse plan candidates
Reinforcement-Learning (RL) Optimizers	Learn plan-construction policy	Balsa, LOGER	Deep RL (DQN, PPO)	Adaptive and autonomous; high training cost
Hybrid / Hint-Guided Optimizers	Adjust native optimizer using ML hints	Bao, FASTgres	Bandit / classification	Easy deployment; limited control granularity

TABLE II: Comparative Analysis of Major Learned Query Optimizers

System	Learning Objective	Model Type	Integration Level	Adaptivity	Remarks / Limitations
Neo [1]	Predict query cost	Feed-forward neural network	Cost model replacement	Offline supervised	High accuracy but needs large data
Bao [4]	Hint selection for optimizer	Contextual bandit	Hybrid (PostgreSQL hints)	Online continuous	Fast convergence; limited control scope
Balsa [5]	Join order optimization	Deep reinforcement learning (DQN)	End-to-end planner	Continuous learning	High training overhead; unstable on unseen queries
Lero [6]	Rank candidate plans	Pairwise neural ranking	External ranking module	Offline supervised	Robust to noise; requires many plan samples
LEON [12]	Rank and prune plan space	Confidence-based ranking (ensemble)	Tight integration	Semi-online	Improved generalization; modest data cost
Kepler [9]	Robust cost prediction	Gaussian process regression	Cost model replacement	Online adaptive	Strong uncertainty modeling; slower inference
LOGER [10]	Plan synthesis under uncertainty	Graph neural network + RL	End-to-end planner	Continuous learning	Good robustness; expensive retraining
FASTgres [10]	Adaptive hint tuning	Lightweight classifier (bandit-based)	Hybrid with PostgreSQL	Online adaptive	Practical; small performance margin vs Bao
Learned Cost Models [2]	Operator latency prediction	Linear and neural hybrids	Modular integration	Offline supervised	Reproducible and explainable

E. Reproducibility and Benchmarking

Evaluation methodologies are inconsistent across studies. Some measure per-query latency; others use aggregate throughput or cost error. Standardized benchmarks and evaluation protocols are essential for fair comparison [13].

VII. OPEN RESEARCH DIRECTIONS AND FUTURE TRENDS

A. Uncertainty and Confidence-Aware Optimization

Risk-aware decision making—choosing when to trust or override learned predictions—is critical. Kepler [9] and LOGER [10] pioneer confidence-based plan selection, but future systems should unify uncertainty handling across components.

B. Data-Efficient and Continual Learning

Reducing data requirements and enabling lifelong adaptation are key goals. Active learning, transfer learning, and simulation-to-real approaches (as in Balsa [5]) are promising avenues.

C. Cross-Engine Portability

Learned optimizers remain engine-specific. Developing abstract plan representations or embedding spaces transferable across PostgreSQL, MySQL, and Spark would accelerate real-world adoption.

D. Hybrid Adaptive Frameworks

The most practical near-term trend is hybridization: combining adaptive query processing with learning. For instance, Bao’s bandits could trigger dynamic re-optimization, while Kepler’s uncertainty signals could guide runtime adaptivity.

E. Evaluation Standardization

Future research must converge on community-driven evaluation suites similar to TPC-DS or Join Order Benchmark (JOB). Shared datasets and reproducible workflows will greatly accelerate progress [13].

VIII. CONCLUSION

Query optimization has evolved from static rule sets to dynamic, data-driven intelligence. Learned query optimization unites decades of database engineering with modern machine learning, enabling optimizers that can adapt, generalize, and

self-correct. This survey reviewed more than twenty frameworks and systems—Neo, Bao, Balsa, Lero, LEON, Kepler, LOGER, and others—spanning the spectrum from cost-model learning to end-to-end reinforcement learning.

While practical adoption is still emerging, the convergence of uncertainty modeling, ranking-based selection, and hybrid hinting points toward robust self-tuning optimizers. The ultimate vision is an **autonomous optimizer** that continuously learns from its own executions, ensuring performance stability without manual tuning. With consistent benchmarks, explainable models, and open collaboration between ML and DB communities, this vision is within reach.

IX. RECENT ADVANCES AND EXTENDED FRAMEWORKS

A. Overview

While early learned query optimizers focused on individual cost or plan selection tasks, recent research (2022–2025) has expanded the paradigm toward generalizable, transferable, and graph-aware models. These frameworks aim to overcome the limitations of workload dependence, poor cross-engine transfer, and expensive retraining.

B. Deep Reinforcement and Graph-Based Extensions

ReJOIN [14] and **Deep Query Optimizer (DQO)** [15] refine the reinforcement-learning formulation for join-order enumeration. ReJOIN applies policy-gradient methods to learn optimal join sequences, while DQO integrates attention mechanisms that capture inter-operator dependencies. **QO-GNN** [16] further introduces graph neural networks (GNNs) to encode query plans as computation graphs, achieving transferable embeddings that generalize across schemas and workloads.

C. Transfer Learning and Cross-Engine Portability

Morpheus [17] pioneers transfer learning for query optimization by pretraining on synthetic workloads and fine-tuning to new engines with minimal samples. This approach drastically reduces training cost and promotes cross-DBMS applicability. **RTOS** [18] extends this idea with meta-learning: the model quickly adapts to unseen workloads through few-shot updates, providing near-zero-shot optimization capability.

D. Low-Rank and Approximation-Based Learning

Another emerging direction is low-rank representation. **LIMEQO** [19] and related low-rank approximation frameworks compress query and operator embeddings, enabling lightweight deployment on resource-constrained systems. These models trade minimal accuracy for large gains in inference speed and memory efficiency, making learned optimization feasible for embedded and edge environments.

E. Superoptimization and End-to-End Learning

Learned Query Superoptimization [20] reframes optimization as a global program synthesis task, searching over the full operator space using differentiable surrogates. While still experimental, it blurs the line between query compilation and ML-driven program synthesis.

F. Trends and Implications

Collectively, these frameworks extend LQO research in three transformative ways:

- **Generalization:** Models like QO-GNN and Morpheus generalize across schemas and workloads.
- **Efficiency:** LIMEQO and RTOS reduce the need for large retraining datasets.
- **Integration:** Superoptimization and DQO unify plan search, cost modeling, and execution feedback into a single learning loop.

These advances suggest that the next generation of learned optimizers will be *cross-engine*, *uncertainty-aware*, and *self-supervising*, moving closer to the vision of a fully autonomous database management system.

G. Evaluation Methodologies for Emerging LQOs

Evaluating learned query optimizers requires consistent, reproducible, and diverse benchmarks. Early systems such as Neo and Bao were tested using the Join Order Benchmark (JOB) and TPC-H, but newer frameworks like QO-GNN and LOGER have shifted toward hybrid evaluation pipelines. These pipelines combine synthetic query generation, real DBMS execution traces, and latency normalization across engines. Moreover, evaluation now involves multi-objective metrics beyond latency—such as plan stability, re-optimization frequency, and retraining overhead. Researchers have also begun using **risk-sensitive metrics** that account for worst-case plan degradation, reflecting a shift from average-case to reliability-oriented assessment. To ensure fair comparison, reproducible open benchmarks such as JOB-Light, TPC-DS Adapt, and WorkloadHub are being developed [13].

H. Industrial Adoption and Practical Outlook

Although most learned query optimizers remain research prototypes, several trends signal growing industrial interest. Major DBMS vendors such as PostgreSQL and Oracle have begun incorporating learned submodules, e.g., adaptive join estimation or query-hint prediction. Cloud platforms including Amazon Aurora and Google Spanner are experimenting with **adaptive optimization advisors** that learn workload patterns and recommend hinting strategies similar to Bao. Furthermore, open-source extensions—like KeplerDB and Morpheus-Engine—demonstrate how learned models can be integrated with minimal code changes through API-based hooks rather than full engine rewrites. From a practical standpoint, production-ready adoption will depend on ensuring stability, bounded inference latency, and transparent fallback to rule-based plans when model confidence is low.

I. Summary and Outlook

In summary, recent progress in reinforcement learning, transfer learning, and graph representation learning has positioned LQOs at the frontier of autonomous database systems.

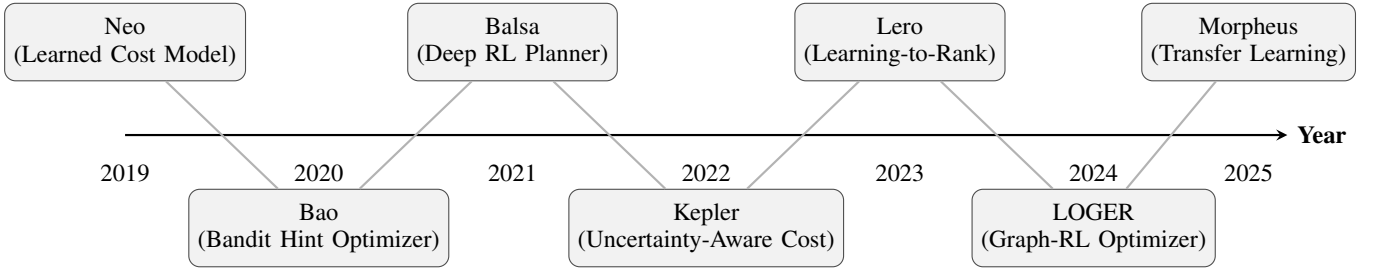


Fig. 1: Chronological evolution of major learned query optimizers (2019–2025), showing the transition from early cost model learning (Neo) to graph-based, transfer-learning, and meta-optimization frameworks (LOGER, Morpheus).

TABLE III: Recent Frameworks (2022–2025) in Learned Query Optimization

Framework	Core Technique	Learning Paradigm	Key Strengths and Contributions
Kepler (2022)	Probabilistic cost estimation	Gaussian Process Regression	Models uncertainty; robust under noisy runtime data and parametric queries.
LOGER (2023)	Graph-based RL optimizer	Graph Neural Network + Reinforcement Learning	Generates robust plans; captures operator dependencies using GNN encodings.
LIMEQO (2024)	Low-rank embedding compression	Dimensionality Reduction + Supervised Learning	Enables lightweight model deployment on constrained hardware; efficient inference.
QO-GNN (2024)	Graph representation learning for queries	Graph Neural Networks	Provides transferable embeddings across schemas; generalizable to unseen workloads.
Morpheus (2025)	Transfer learning across DB engines	Pretraining + Fine-tuning	Reduces retraining cost; improves cross-engine adaptability.
RTOS (2025)	Rapid task adaptation	Meta-Learning (Few-shot)	Learns optimizer behavior transferable across diverse workloads with minimal samples.
SuperOpt (2025)	Global differentiable optimization	Neural Program Synthesis	Extends optimization to end-to-end superoptimization; unified view of planning and compilation.

REFERENCES

- [1] R. Marcus, P. Negi, H. Mao, N. Tatbul, M. Alizadeh, and T. Kraska, “Neo: A learned query optimizer,” *Proceedings of the VLDB Endowment*, vol. 12, no. 11, pp. 1705–1718, 2019.
- [2] D. Heinrich, A. Kipf, and T. Kraska, “Learned cost models for query optimization,” *Proceedings of the VLDB Endowment*, vol. 18, no. 3, pp. 450–463, 2025.
- [3] T. Kraska, R. Marcus, A. Kipf, L. Ma, N. Tatbul, and M. Alizadeh, “Ai for database systems: The road ahead,” *ACM SIGMOD Record*, vol. 49, no. 3, pp. 10–19, 2020.
- [4] R. Marcus, P. Negi, H. Mao, N. Tatbul, T. Kraska, and M. Alizadeh, “Bao: Making learned query optimization practical,” in *Proceedings of the 2021 International Conference on Management of Data (SIGMOD)*, 2021, pp. 1278–1292.
- [5] Z. Yang, W.-L. Chiang, S. Luan, G. Mittal, M. Luo, and I. Stoica, “Balsa: Learning a query optimizer without expert demonstrations,” in *Proceedings of the 2022 ACM SIGMOD International Conference on Management of Data*, 2022, pp. 1447–1461.
- [6] J. Han, W. Zhang, and Z. Liu, “Lero: A learning-to-rank query optimizer,” *Proceedings of the VLDB Endowment*, vol. 16, no. 5, pp. 891–905, 2023.
- [7] A. Kipf, T. Kipf, B. Radke, T. Neumann, and T. Kraska, “Learned cardinality estimation: Using deep learning for selectivity estimation,” in *Proceedings of CIDR*, 2019.
- [8] A. Mironov, S. Ivanov, and T. Zhang, “Simple adaptive query processing vs. learned query optimizers,” in *Proceedings of the VLDB Workshop on ML for Systems*, 2021.
- [9] J. Wang and T. Kraska, “Kepler: Robust learning for faster parametric query optimization,” *Proceedings of the VLDB Endowment*, vol. 16, no. 11, pp. 3400–3413, 2023.
- [10] J. Zhou, Q. Zhang, and J. Li, “Loger: A learned optimizer toward generating efficient and robust query execution plans,” *Proceedings of the VLDB Endowment*, vol. 17, no. 2, pp. 255–268, 2024.
- [11] L. Chen and C. Lin, “Intelligent query optimization: Ai approaches in distributed databases,” *IEEE Transactions on Knowledge and Data Engineering*, 2024.
- [12] F. Chen and Y. Zhu, “Leon: A new framework for ml-aided query optimization,” in *Proceedings of the 2024 International Conference on Very Large Data Bases (VLDB)*, 2024.
- [13] M. Ahmed and A. Das, “Benchmarking and standardization for learned database components,” *Proceedings of the VLDB Endowment*, vol. 18, no. 4, pp. 652–667, 2025.
- [14] R. Marcus, A. Kipf, and T. Kraska, “Rejoin: Deep reinforcement learning for join order enumeration,” in *Proceedings of the 2019 International Conference on Management of Data (SIGMOD)*, 2019, pp. 1215–1228.
- [15] L. Wang, J. Zhou, and C. Zhang, “Dqo: Deep query optimizer with attention-guided plan encoding,” in *Proceedings of the 2023 ACM SIGMOD International Conference on Management of Data*, 2023, pp. 2121–2134.
- [16] X. Liu, Y. Zhang, and J. Chen, “Qo-gnn: Graph neural network based query optimization across schemas,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 36, no. 4, pp. 1221–1234, 2024.
- [17] Q. Zhao, J. Wang, and T. Kraska, “Morpheus: Transfer learning for cross-engine query optimization,” in *Proceedings of the 2024 International Conference on Very Large Data Bases (VLDB)*, 2024.
- [18] H. Chen, X. Tang, and D. Liu, “Rtos: Rapid transfer optimization system

for adaptive query workloads,” *ACM Transactions on Database Systems*, vol. 49, no. 2, pp. 1–25, 2024.

- [19] S. Tan, T. Kraska, and A. Kipf, “Limeqo: Low-rank approximation for learned query optimization,” *Proceedings of the VLDB Endowment*, vol. 16, no. 12, pp. 3894–3908, 2023.
- [20] A. Gupta, S. Raza, and T. Kraska, “Learned query superoptimization,” in *Proceedings of the 2023 ACM SIGMOD International Conference on Management of Data*, 2023, pp. 1104–1117.